
Investigating Component Contributions in Multi-Agent ML Systems

Junsung Kim^{*1} Ilia Mireskandari^{*1} Seungwan Son^{*1} Yifan Zhou^{*1} Khizer Shahid^{*1} Dylan Yi Han Dai^{*1}

Abstract

Autonomous agents for machine learning engineering have advanced rapidly, yet comparing their effectiveness remains difficult. Existing systems combine different techniques—multi-agent decomposition, iterative refinement, memory management, and planning—in varying configurations, making it unclear which components actually drive performance. Complicating evaluation, existing benchmarks rely on historical competitions whose data likely contaminates LLM training corpora and whose static baselines reflect outdated human performance. To address this, we conduct approximately 4,000 controlled experiments systematically ablating architectural components, alongside K-LIVE¹ a new benchmark of 25 competitions that provides a dynamic evaluation environment with minimal data contamination. Our findings challenge common design assumptions: in our evaluation, iterative feedback contributes more than architectural complexity, and fixed-role multi-agent coordination consistently underperforms a single-agent baseline. These results provide concrete guidance for practitioners building ML engineering agents.

1. Introduction

Autonomous agents for machine learning engineering—systems that analyze datasets, engineer features, select models, and produce competitive predictions—have advanced rapidly. Systems like AIDE (Jiang et al., 2025), DS-Agent (Guo et al., 2024), and AutoKaggle (Li et al., 2024) demonstrate that LLM-based agents can achieve non-trivial performance on ML benchmarks. These systems differ along several architectural choices, including whether they use multi-agent decomposition, iterative refinement, structured

memory, explicit planning, and retrieval. Yet as these systems proliferate, a fundamental problem has emerged: we lack systematic understanding of which design choices actually matter.

Because these choices are bundled differently across systems (Table 1), end-to-end performance comparisons provide little causal evidence about individual components. Multi-agent frameworks such as AutoGen (Wu et al., 2024) and MetaGPT (Hong et al., 2024), and memory-augmented agents such as MemGPT (Packer et al., 2023), illustrate the range of design patterns now used in agent systems. A strong result from one agent could reflect its base model, prompting style, retrieval setup, iteration budget, or coordination protocol rather than the architectural feature being emphasized. This makes it difficult to determine whether increasingly complex agent designs are necessary, or whether simpler iterative baselines capture most of the benefit.

Complicating evaluation, existing benchmarks present methodological concerns. MLE-Bench (Chan et al., 2025) and MLE-Dojo (Qiang et al., 2025) rely on historical Kaggle competitions. This raises two critical issues: (1) competition data likely contaminates LLM training corpora, making it unclear whether agents are reasoning or recalling; and (2) winning solutions from years past set a static, often outdated performance bar.

This paper addresses both questions. We conduct approximately 4,000 controlled experiments systematically ablating five architectural components across 75 historical competitions (MLE-Bench) and 25 active competitions (K-LIVE, introduced here). Evaluating on both allows us to test whether findings from historical benchmarks generalize to contamination-minimal settings with contemporary human baselines.

Our analysis yields several findings that challenge prevailing design intuitions. Iterative refinement provides larger gains than any other component: more than multi-agent coordination, memory, or planning. Fixed-role multi-agent systems underperform single-agent baselines, with failure traces consistent with coordination overhead. Findings replicate on K-LIVE despite its live, minimal contamination setting, suggesting that the architectural patterns are not artifacts of historical benchmark memorization.

^{*}Equal contribution ¹Celestra. Correspondence to: Junsung Kim <junsung2@andrew.cmu.edu>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹<https://github.com/mireskandari/klive>

Table 1. Architectural components across ML engineering agents. Systems achieve comparable performance despite different designs, making it difficult to attribute success to specific components.

System	Multi-Agent	Iterative	Memory	Planning	Retrieval
AIDE (Jiang et al., 2025)	✗	✓	✓	✓	✗
OpenHands (Wang et al., 2025)	✗	✓	✗	✗	✗
MLAgentBench (Huang et al., 2024)	✗	✓	✗	✗	✗
AutoKaggle (Li et al., 2024)	✓	✓	✓	✓	✓
DS-Agent (Guo et al., 2024)	✗	✓	✓	✗	✓
MetaGPT (Hong et al., 2024)	✓	✓	✓	✓	✗
Data Interpreter (Hong et al., 2025)	✗	✓	✗	✓	✗
MLCopilot (Zhang et al., 2024)	✗	✗	✓	✗	✓

2. Preliminary

ML engineering agents require evaluation approaches that differ from standard code generation benchmarks. Unlike code completion, where success means producing syntactically correct output, ML engineering involves open-ended problem solving: data exploration, hypothesis formation, implementation, debugging, and iterative improvement. This section reviews existing evaluation approaches and their limitations.

Kaggle as evaluation platform. Kaggle competitions have emerged as the primary testbed for ML engineering agents. Competitions provide objective metrics, real-world datasets, and human baselines for comparison. More importantly, Kaggle problems require the full ML workflow—data exploration, feature engineering, model selection, hyperparameter tuning, and submission formatting. Alternative benchmarks evaluate narrower skills: MLAgentBench (Huang et al., 2024) tests hyperparameter tuning on fixed codebases, while DS-1000 (Lai et al., 2023) evaluates isolated data science code completion. Neither captures end-to-end ML engineering capability.

MLE-Bench. The most prominent Kaggle-based benchmark, MLE-Bench (Chan et al., 2025), curates 75 historical competitions spanning diverse domains with standardized evaluation infrastructure: isolated Docker environments, Kaggle API integration, and medal-based scoring. However, relying on historical data introduces two limitations.

Training data contamination. Historical MLE-Bench competitions are extensively documented online—winning notebooks, discussion threads, and post-competition writeups appear across the web. This content likely exists in modern LLM training corpora, making it difficult to distinguish genuine reasoning from pattern recall when an agent performs well.

Stale human baselines. Historical leaderboards reflect techniques available at competition time. For example, a 2019 gold-medal solution for an image classification task relied

on older models from the ResNet family². These competitions are easily medal-achievable by utilizing the same family model that has been released later. Agents evaluated against these frozen targets compete with outdated human performance.

These limitations motivate K-LIVE, which evaluates on active competitions to ensure contamination-minimal assessment against contemporary human baselines.

3. The K-LIVE Benchmark

K-LIVE addresses the limitations outlined above by shifting evaluation to active, ongoing Kaggle competitions. By selecting the most recent ongoing competitions, we minimize the risk of competition-specific information appearing in LLM pre-training corpus. These competitions are evaluated on persistently updating leaderboards, so agents will compete against live competitors using state-of-the-art research and development.

3.1. Competition Selection

We curated 25 competitions according to three criteria, each addressing a distinct experimental requirement.

Computational tractability. Our ablation study requires thousands of experimental runs across dozens of agent configurations. To make this feasible, each competition must be tractable within a 24-hour window on standardized hardware.

Evaluation integrity. Some competitions suffer from label leakage, including test labels discoverable through metadata, public datasets, or reverse engineering. Including such competitions would confound our measurement: an agent might score well by exploiting leakage rather than solving the underlying ML problem. We manually audited each candidate and excluded those with known integrity issues.

Domain breadth. Existing benchmarks skew heavily toward

²<https://www.kaggle.com/competitions/aptos2019-blindness-detection>

Table 2. Model cutoff snapshot for the four backbones evaluated on K-LIVE Tier 1. “Official” dates are published by the provider; “Proxy” dates use the model release date as a surrogate.

Model	Cutoff type	Date	Evidence
Gemini 3 Flash Preview	Knowledge	Jan 2025	Official
DeepSeek-V3.2	Knowledge	Mar 2025	Official
Claude Sonnet 4.6	Knowledge	Aug 2025	Official
Kimi K2.5	Proxy	Jan 2026	Release-date

tabular classification and standard NLP tasks. If our ablation findings only held for these domains, their generality would be limited. We deliberately selected competitions spanning 11 categories—including 3D volumetric segmentation, biological sequence prediction, and sports analytics—to ensure our architectural conclusions transfer across problem types. The full domain distribution appears in Table 15.

The complete competition list, including Kaggle slugs, data types, team counts, and difficulty ratings, is provided in Tables 13 and 14.

3.2. Two-Tier Structure

K-LIVE organizes its 25 competitions into two tiers serving complementary purposes (Section A.2).

Tier 1 contains 13 active competitions with deadlines. These provide the strongest defense against training contamination: the datasets were released after current model knowledge cutoffs, and the community is still actively exploring solution strategies. As competitions conclude, we rotate in new ones, making K-LIVE a rolling benchmark that remains current.

Tier 2 contains 12 perpetual competitions that remain fixed across benchmark versions. While these problems are older and have public notebooks available, we selected tasks where execution quality matters more than strategy knowledge—problems where knowing “use XGBoost” provides little advantage without proper implementation, debugging, and tuning. Tier 2 enables longitudinal comparison: researchers can track agent improvement over time against a stable baseline.

3.3. Contamination Analysis

We assess Tier 1 contamination risk by comparing each competition’s start date against the available cutoff evidence for each evaluated backbone. This check does not establish a full contamination-free guarantee, since training corpora are not fully disclosed, but it directly addresses the main temporal route by which benchmark content could enter pretraining. Table 2 summarizes the cutoff evidence used in this analysis.

Table 3. Tier 1 cutoff evaluation matrix. **Pass**: Competition started after the cutoff. **Fail**: Started before. **Unc.**: Same month.

Cutoff reference	Pass	Fail	Unc.
Gemini 3 Flash Preview (Know.: Jan ’25)	13	0	0
DeepSeek-V3.2 (Know.: Mar ’25)	13	0	0
Claude Son. 4.6 (Know.: Aug ’25)	13	0	0
Kimi K2.5 (Proxy: Jan ’26)	0	8	5

Table 3 summarizes this comparison across all Tier 1 competitions. Every Tier 1 competition starts after the reported cutoff for the tested models except Kimi K2.5, where the available reference is a release-date proxy rather than a disclosed knowledge cutoff. This benchmark is strongest for backbones whose disclosed cutoff references predate the active competition set. The proxy-only cases are reported separately in the matrix.

3.4. Evaluation Metric

We tracked each agent’s K-LIVE ranking for 30 days after submission. Performance is reported as public-leaderboard percentile rank: for rank R among N teams,

$$\text{Percentile} = 100 \times \frac{N-R+1}{N}.$$

We use percentile rank instead of medal thresholds for two reasons. First, it gives a continuous measure of performance: a 5-point gap between two configurations remains visible, whereas medal counts may hide it. Second, it normalizes across competitions of different sizes and difficulty. For example, the 80th percentile represents strong relative performance in both a 5,000-team competition and a 200-team competition, even though the absolute score required may differ. Details on versioning and leaderboard snapshots are given in Section A.7.

3.5. Validating the Benchmark Gap

To quantify the difference between historical and live evaluation, we ran leading ML engineering agents on both MLE-Bench and K-LIVE. Table 4 reports the results. MLE-Bench scores reflect medal rates (percentage of competitions achieving bronze or better); K-LIVE scores reflect mean percentile rank across the 25 competitions. Higher medal rate does not always correlate with stronger percentile performance, as the metrics capture different aspects of capability.

The comparison shows a coarse capability ordering: the strongest systems on MLE-Bench generally remain among the strongest on K-LIVE, and the weakest systems remain weak. However, pairwise rankings are not identical, reflecting both benchmark differences and the fact that medal rates and percentile ranks measure different aspects of perfor-

Table 4. Performance comparison between MLE-Bench (historical) and K-LIVE (live). Higher is better for both metrics. Agent citations in row order: (Chopde et al., 2025; Li et al., 2025; Liu et al., 2025; Li et al., 2024; Guo et al., 2024; Hong et al., 2024; Jiang et al., 2025; Wang et al., 2025).

Agent	LLM	MLE-Bench	K-LIVE
PiML	Gemini-3-Pro	61.3%	91.1 %
Famou-Agent 2.0	Gemini-2.5-Pro	59.6%	81.8 %
ML-Master 2.0	DeepSeek-V3.2	56.4%	84.7 %
AutoKaggle	GPT-4o	43.2%	65.6 %
DS-Agent	GPT-4o	38.7%	59.3 %
MetaGPT	GPT-4o	35.1%	63.8 %
AIDE	o1-preview	17.1%	38.2 %
OpenHands	GPT-4o	4.9%	24.3 %

mance. Moreover, the metrics are not directly comparable—medal rates and percentile ranks measure different aspects of performance. The percentile threshold to obtain a medal in Kaggle varies by the number of competitors in a competition, making medal counts an unreliable variable to measure objective performance of the agent when compared to human competitors. What the comparison does reveal is that K-LIVE successfully discriminates between agents across the full capability spectrum, validating K-LIVE as a meaningful evaluation while providing contamination safeguards that historical benchmarks cannot offer.

4. Experimental Setup

The systems in Table 1 achieve comparable performance despite different architectures, making it difficult to attribute success to specific design choices. We address this through controlled ablation: building a configurable agent and systematically varying its components to isolate individual contributions.

Approach. Rather than proposing a new system, we construct a modular agent architecture where each component can be independently enabled or disabled. Our baseline reflects current best practice—a single agent with iterative refinement, similar to AIDE (Jiang et al., 2025). We then measure the effect of adding or removing each component while controlling for confounds like base model capability and compute budget.

The baseline. Figure 1 shows the baseline agent (C1). Given a competition, the agent (1) reads the problem description and data, (2) generates Python code to train a model and produce predictions, (3) executes the code and observes results, (4) revises based on feedback, and (5) repeats steps 2–4 for up to 10 iterations. Retrieval is enabled, allowing the agent to access public Kaggle notebooks and datasets. This configuration mirrors deployed systems. Fig-

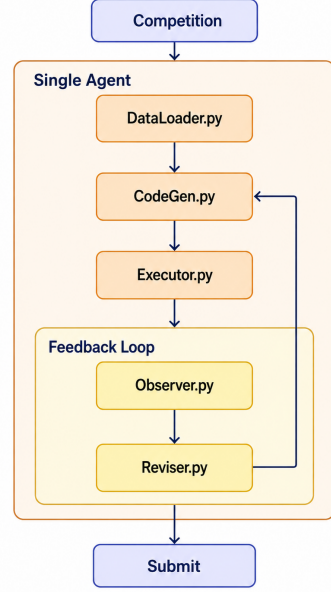


Figure 1. Baseline configuration (C1). A single agent iteratively generates code, executes, observes results, and revises. The feedback loop (yellow) enables up to 10 revision cycles—our ablation identifies this as the dominant performance factor.

ure 2 shows the maximum-complexity configuration (C16) with all components enabled.

Components under study. We selected five components based on the architectural differences in Table 1:

- **Iterative refinement.** The agent observes execution feedback (errors, metrics, leaderboard scores) and revises its approach. We compare 1 iteration (no feedback) against 3, 5, 7 and 10 iterations to measure whether debugging and incremental improvement—central to human ML workflows—transfer to agents.
- **Multi-agent coordination.** Multiple specialized agents divide the work: an Explorer analyzes data, a Builder writes training code, and an Evaluator checks for errors before submission. Agents communicate through structured messages. This tests whether task decomposition improves performance on complex problems.
- **Structured memory.** The agent maintains an explicit log of past experiments, recording what was tried, what metrics resulted, and what errors occurred. The agent queries this log when planning new experiments, testing whether structured history provides value beyond raw context.
- **Explicit planning.** Before writing code, the agent generates a structured plan: problem type, likely challenges, promising approaches, and experiment sequence. This

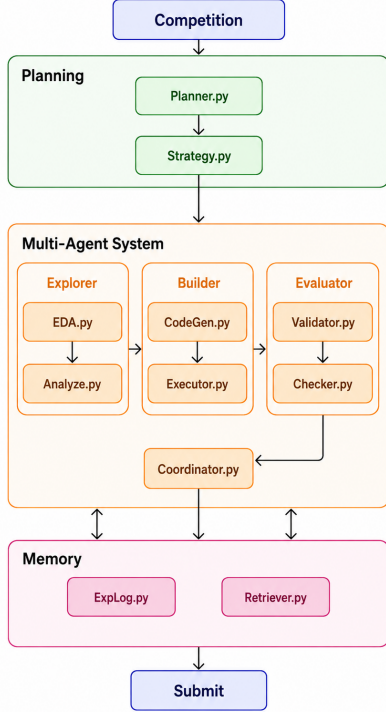


Figure 2. All-components configuration (C16). Adds planning, multi-agent coordination, and memory to the baseline. Despite architectural sophistication, this configuration performs 13.7 points worse than C1 while consuming $2.9\times$ as many tokens.

tests whether deliberation before action improves outcomes.

- **Retrieval.** The agent can access public Kaggle notebooks and datasets. This disables retrieval forces reliance on parametric knowledge alone, and isolates the contribution of external knowledge access.

Configurations. Table 5 shows the 16 configurations we test. Single-component ablations (C1–C6) isolate individual effects. Because iteration emerged as the dominant factor in preliminary experiments, we specifically test how other components interact with iteration depth (C7–C11). Component combination studies (C12–C15) test whether components synergize. The all-components configuration (C16) tests maximum complexity.

Evaluation. We run experiments on 100 competitions: 75 from MLE-Bench (Chan et al., 2025) and 25 from K-LIVE. Each configuration runs three independent replicates to account for LLM sampling variance. Each run has a 24-hour time budget on one A100 GPU. The primary 16-configuration ablation uses DeepSeek-V3.2 (DeepSeek-AI, 2025) to isolate architectural effects from model capability. For backbone-generality checks, we additionally reran C1–C5 and C16 on Kimi K2.5, Claude Sonnet 4.6, and Gemini

Table 5. The 16 configurations in our ablation study.

ID	Configuration	Purpose
<i>Single-component ablations</i>		
C1	Baseline (10 iter, retrieval)	Reference
C2	No iteration (1 round)	Iteration effect
C3	+ Multi-agent	Multi-agent effect
C4	+ Memory	Memory effect
C5	+ Planning	Planning effect
C6	No retrieval	Retrieval effect
<i>Iteration interactions</i>		
C7	Multi-agent + 3 iter	Does multi-agent need less iteration?
C8	Multi-agent + 5 iter	
C9	Memory + 5 iter	
C10	Planning + 5 iter	
C11	Planning + 3 iter	
<i>Component combinations</i>		
C12	Multi-agent + Memory	Synergy tests
C13	Multi-agent + Planning	
C14	Memory + Planning	
C15	Multi + Memory + Planning	
C16	All components	Max complexity

3 Flash Preview. We measure percentile rank on the public leaderboard, which provides continuous granularity and adjusts for competition difficulty. The primary ablation completed 3,973 successful experiments; the cross-model checks are reported separately.

5. Results

We structure our analysis to progressively narrow down what matters. We first establish which individual components affect performance (§5.1). The results reveal iteration as the dominant factor, so we investigate its scaling behavior (§5.2). We then test whether other components can substitute for iteration (§5.4) or combine beneficially (§5.6). Finally, we analyze failure patterns (§5.7) and validate that findings generalize to live evaluation (§5.8).

5.1. Which Components Matter?

We begin by measuring each component’s individual effect on the baseline. Table 6 shows the results.

Iteration dominates. Removing it (C2) causes a 33.7-point drop on MLE-Bench and 34.1-point drop on K-LIVE—larger than all other effects combined. This establishes that the ability to observe feedback and revise is the single most important capability.

The fixed-role Explorer–Builder–Evaluator pipeline (C3) hurts performance: -8.3 points on MLE-Bench, -8.5 on K-LIVE. This pattern suggests that task decomposition through this protocol adds coordination costs that outweigh

Table 6. Single-component ablation results.

Configuration	MLE-Bench (%)		K-LIVE (pctl)	
	Score	Δ	Score	Δ
C1: Baseline	52.4	-	81.3	-
C2: No iteration	18.7	-33.7	47.2	-34.1
C3: + Multi-agent	44.1	-8.3	72.8	-8.5
C4: + Memory	53.8	+1.4	82.1	+0.8
C5: + Planning	53.1	+0.7	81.9	+0.6
C6: - Retrieval	41.2	-11.2	—	—

any benefit from specialization.

To test whether this pattern is specific to DeepSeek-V3.2, we reran the key component-isolation configurations on three additional frontier backbones.

Table 21 shows that stronger backbones improve single-agent performance, but not fixed-role multi-agent coordination. C1 rises from 81.3 with DeepSeek-V3.2 to 86.3–88.9 with the other backbones, while C3 remains confined to a narrow 71.4–73.3 band. Consequently, the multi-agent penalty widens on the stronger backbones rather than shrinking, suggesting that coordination overhead is a structural property of fixed-role sequential pipelines rather than a base-model capability deficit. The C2 results separate backbone capability from coordination: stronger models substantially improve the no-iteration setting, shrinking the C1–C2 gap from 34.1 points for DeepSeek-V3.2 to 25.1–26.3 points for the other backbones, yet they do not improve C3, which remains near 71–73 percentile.

We refer to this pattern as a coordination ceiling: in our experiments, the fixed-role sequential pipeline appears to impose an upper bound on multi-agent performance (71–73 percentile) that does not rise with backbone capability. This observation is consistent with the bottleneck being structural rather than capability-driven, though we note it is based on one coordination protocol across four models and may not generalize to all multi-agent designs.

Memory (C4) and planning (C5) provide small gains in performance; however, given the added complexity they introduce, the practical benefit is questionable.

Retrieval (C6) matters for historical competitions (−11.2 points without it). We restrict the retrieval ablation to MLE-Bench because retrieval availability is not comparable across K-LIVE tiers: Tier 1 active competitions have sparse public materials, while Tier 2 perpetual competitions have public notebooks.

5.2. How Much Does Iteration Help?

Given iteration’s dominance, we examine its scaling behavior at different iteration counts in Table 7.

Table 7. Baseline performance improves with diminishing returns as iterations increase.

Iter	MLE (%)	K-LIVE (pctl)	Tokens (M)	\$/run
1	18.7	47.2	1.10	0.18
3	38.4	65.8	1.16	0.19
5	45.6	74.1	1.84	0.31
10	52.4	81.3	3.52	0.59

Performance shows diminishing returns: the jump from 1→3 iterations (+19.7 points MLE) exceeds the gain from 3→10 (+14.0 points). This creates a practical tradeoff: 3 iterations achieve 73% of the 10-iteration MLE score and 81% of the 10-iteration K-LIVE score at roughly one-third of the cost.

5.3. What Are Iterations Actually Doing?

The +33.7 point gain from 1 to 10 iterations (§5.2) comes mostly from fixing broken code, not from searching for better ideas. Of 500 sampled runs, 58% spend at least one iteration recovering from a runtime or resource failure before they produce any valid submission (Table 33), and most failures are technical problems—code errors, format mismatches, and out-of-memory—rather than the agent being unable to solve the task (Table 29). The completion data shows the same thing: turning iteration on drops the no-submission rate from 22.7% (C2) to 10.3% (C1), so many failed runs would have succeeded with a single retry. Most of the gain also arrives early—the 1→3 step alone gives +19.7 points—after which later iterations mostly do small tuning. Our reading is that iteration mainly buys robustness against the buggy first drafts current LLMs tend to write. If so, much of its value could be had far more cheaply (Appendix D.9) with simple pre-execution checks such as linting, validating the submission format, and a quick one-batch test run, leaving the iteration budget for real optimization.

5.4. Can Other Components Substitute for Iteration?

A natural hypothesis: perhaps multi-agent coordination, memory, or planning can achieve similar results with fewer iterations. If specialized agents communicate effectively, they might need less trial-and-error. We test this by combining components with reduced iteration counts and comparing against the single-agent baseline at matched iterations.

Table 22 shows that no tested component substitutes for iteration. Fixed-role multi-agent coordination hurts by the same margin (≈ 9 points) regardless of iteration count—it does not become more useful when iterations are limited. Memory and planning provide no clear benefit at reduced iterations. In this setting, performance improves most reliably by preserving the execution-feedback loop.

5.5. Iso-Cost Comparison

A natural critique of comparing single-agent and multi-agent configurations at fixed iteration counts is that multi-agent systems consume significantly more tokens due to inter-agent communication overhead. Does the fixed-role multi-agent pipeline underperform simply because it exhausts its token budget on coordination rather than problem-solving?

To address this, we re-examine our results under an iso-cost lens. Table 8 aligns configurations by their total token consumption (cost) rather than their iteration count. A fully extended cost-tier breakdown is available in Section D.10.

Table 8. Iso-cost comparison. The single-agent baseline (C1) vastly outperforms multi-agent setups at both similar and strictly higher token budgets.

Config	Iter	Tok(M)	MLE%	K-LIVE
<i>Lower/Similar Budget</i>				
C7: Multi	3	2.9	29.1	56.8
C1: Base	10	3.5	52.4	81.3
C8: Multi	5	4.7	36.2	65.1
<i>Higher Budget</i>				
C3: Multi	10	7.4	44.1	72.8

The data reveals that the multi-agent penalty is not an artifact of an unfair cost comparison. The single-agent baseline (C1) achieves 81.3 percentile on K-LIVE using 3.5M tokens. A multi-agent configuration restricted to a comparable token budget falls between C7 and C8, which only achieve 56.8 and 65.1 percentile.

Furthermore, even when the multi-agent pipeline is granted more than double the token budget of the baseline (C3 at 7.4M tokens), it still underperforms the cheaper single-agent system by 8.5 points on K-LIVE. Providing the single-agent baseline with an equivalent 7.4M token budget (via additional iterations) would only widen this gap further. Therefore, the fixed-role multi-agent architecture tested is strictly dominated on a cost-performance basis.

5.6. Do Components Combine Beneficially?

Perhaps components that are neutral alone become beneficial together. We test all pairwise and triple combinations at full iteration count.

Table 9 shows no positive interactions. Memory does not rescue multi-agent (C12). Planning does not rescue multi-agent (C13). The triple combination (C15) exhibits negative synergy: it performs 3.1 points worse than predicted from individual effects on MLE-Bench and 3.0 points worse on K-LIVE. The all-components configuration (C16) is the overall worst among the full-iteration combinations: 13.9 points below baseline while consuming $2.9\times$ as many tokens.

Table 9. Component combinations show no positive synergy.

Configuration	Δ MLE	Δ K-LIVE
C1: Baseline	—	—
C12: Multi + Memory	−7.2	−7.9
C13: Multi + Plan	−7.6	−8.4
C14: Memory + Plan	+0.8	+0.4
C15: Multi + Mem + Plan	−9.3	−10.1
C16: All components	−13.7	−13.9

Table 10. Failure rates by configuration. Complex configurations fail more often and on different modes.

Config	Fail%	Primary Mode	Secondary
C1	10.3	Feature eng. (42%)	Debug (31%)
C3	18.0	Coordination (38%)	Format (24%)
C7	23.7	Timeout (41%)	Coord. (29%)
C16	20.7	Coordination (35%)	Timeout (28%)

5.7. Why Do Complex Configurations Fail?

To understand why architectural complexity hurts, we analyze failure patterns across configurations.

Table 10 reveals distinct failure modes. Single-agent configurations (C1) fail when the problem requires sophisticated feature engineering that the agent cannot discover through iteration—a capability limitation. Multi-agent configurations fail on coordination: agents produce incompatible outputs, enter circular revision loops, or miscommunicate. At low iterations (C7), coordination overhead causes timeouts—agents spend tokens negotiating instead of solving. The failure mode shift from capability limits to coordination overhead helps explain why the tested fixed-role pipeline underperforms: it trades some problems the agent cannot solve for coordination problems the pipeline creates.

5.8. Do Findings Generalize to K-LIVE?

A critical question: do our findings on MLE-Bench, which may be contaminated, replicate on K-LIVE? If agents simply memorize solutions for historical competitions, patterns observed on MLE-Bench might not reflect genuine architectural effects.

Table 11 breaks down performance on K-LIVE by tier and domain. The results confirm that our findings generalize.

Iteration remains dominant on both tiers: removing it drops performance by 34 points on Tier 1 (active competitions designed to minimize contamination risk) and 34 points on Tier 2. Multi-agent coordination hurts by 8–9 points on both tiers. Memory and planning remain neutral.

The consistency across tiers is notable. Tier 1 competitions

Table 11. Detailed K-LIVE results by tier. Findings replicate: iteration dominates, multi-agent hurts, memory/planning neutral.

Config	Tier 1 (13)		Tier 2 (12)		Avg
	Pctl	Δ	Pctl	Δ	
C1	79.2	—	83.6	—	81.3
C2	44.8	−34.4	49.8	−33.8	47.2
C3	70.4	−8.8	75.4	−8.2	72.8
C4	79.9	+0.7	84.5	+0.9	82.1
C5	79.6	+0.4	84.4	+0.8	81.9
C16	65.1	−14.1	69.9	−13.7	67.4

are launched after most model knowledge cutoffs, minimizing contamination risk (§3.3). Tier 2 competitions have public notebooks but test execution quality over strategy recall. That both tiers show identical patterns suggests our findings reflect genuine architectural effects, not memorization artifacts.

Table 16 shows K-LIVE performance by problem domain. Iteration dominance holds universally: C1 outperforms C2 by 30+ points across all domains. The multi-agent penalty is smaller on tabular tasks (−6.6 points) and larger on vision tasks (−10.4 points). This aligns with our failure analysis and is consistent with coordination overhead scaling with task complexity.

The K-LIVE results validate two claims. First, our architectural findings are not solely artifacts of benchmark contamination: they replicate on competitions designed to minimize contamination risk. Second, the failure modes on K-LIVE mirror those on MLE-Bench, confirming its utility as an evaluation benchmark.

5.9. Summary

Our ablation yields three findings. First, **iteration dominates**: the ability to execute, observe, and revise provides more value than any architectural sophistication. Second, fixed-role multi-agent coordination consistently underperforms in our evaluation: coordination overhead exceeds specialization benefits. Third, **components do not combine beneficially**: adding complexity reduces performance while increasing cost. These findings replicate across MLE-Bench and K-LIVE, across Tier 1 and Tier 2, and across all problem domains—suggesting they reflect properties of the evaluated task regime and tested implementations rather than benchmark-specific artifacts.

6. Limitations

Our findings come with caveats that bound their generalizability. We study end-to-end supervised ML engineering tasks: data understanding, feature engineering, model selec-

tion, training, and evaluation across 25 Kaggle competitions spanning 11 domains. We treat this as the empirical scope of our study, while noting that it does not cover production deployment, monitoring, maintenance, or reliability workflows.

Primary-backbone design. The 16-configuration ablation uses DeepSeek-V3.2 because the study scale made cost-performance a binding constraint. We mitigate the single-backbone concern by rerunning six high-signal configurations across Kimi K2.5, Claude Sonnet 4.6, and Gemini 3 Flash Preview, where the main pattern persists. This check does not exhaust the model space, so our conclusions apply most directly to frontier-class backbones and to the fixed-role sequential coordination protocol we test.

Fixed coordination protocol. Our multi-agent system uses a specific communication structure: Explorer → Builder → Evaluator with structured message passing. Alternative protocols might reduce coordination overhead. We test one reasonable implementation of each component, not the space of possible multi-agent protocols, memory systems, or planning methods. Accordingly, our negative multi-agent result should be read as evidence about this fixed-role sequential pipeline, not as a claim about all possible multi-agent protocols.

Competition-specific evaluation. Kaggle competitions, while realistic, represent a narrow slice of ML engineering. Production ML involves deployment, monitoring, and maintenance—tasks our evaluation does not capture. Agents optimized for competition performance may not transfer to engineering contexts where reliability matters more than leaderboard rank.

Contamination bounds. Our contamination defense relies on date filtering, but it is not airtight. Date filtering depends on cutoff disclosures that are coarse (month-level) or absent (proxy models), and cannot verify the exact composition of model training corpora. A model could have ingested data structurally similar to a Tier 1 competition even if the competition itself postdates the cutoff. We therefore describe K-LIVE as *designed to minimize contamination risk* rather than contamination-free.

Compute assumptions. We fix budget at 24 hours per run. Under tighter constraints, the calculus may shift: if only 1–2 iterations are feasible, components that frontload quality (planning, retrieval) might matter more. Our finding that iteration dominates assumes sufficient compute to iterate.

7. Discussion

The central finding of this work is negative: in our evaluation, architectural complexity does not improve over a strong iterative single-agent baseline. The fixed-role multi-agent

pipeline underperforms, memory and planning provide at most marginal gains, and the all-components configuration performs worst. This challenges the prevailing assumption that more sophisticated architectures yield better agents.

Why does iteration win? We hypothesize that iterative refinement succeeds because it offloads cognition to the environment. Rather than reasoning about what might work, the agent tries something and observes what actually happens. This empirical feedback is more reliable than internal simulation, especially for ML tasks where small implementation details determine success. The agent need not predict whether a feature will help—it can measure.

The success of iteration also reflects the nature of ML engineering itself. Unlike theorem proving where correct reasoning guarantees success, ML engineering is fundamentally empirical: whether a feature helps or a hyperparameter works is answered by experiment, not deduction. An architecture that embraces this empirical nature (by iterating) outperforms one that tries to reason its way to solutions.

Why does the fixed-role pipeline hurt? In this pipeline, coordination introduces failure modes that the single-agent baseline avoids. When one agent misunderstands another’s output, errors compound. The Explorer might identify a promising feature, but if the Builder misinterprets the specification, it gets implemented incorrectly. With iteration, a single agent catches such errors on the next round. With multiple agents, errors propagate through the pipeline before detection.

Our failure analysis (§5.7) is consistent with this account: multi-agent configurations shift from capability-limited failures (feature engineering) to coordination-limited failures (miscommunication, timeouts). The failure traces suggest that the pipeline trades some capability-limited failures for coordination-limited failures.

Why don’t memory and planning help? The null results for memory and planning are perhaps more surprising. We offer two hypotheses. First, modern LLMs may already perform implicit planning and maintain implicit memory within their context windows, making explicit systems redundant. Second, the overhead of maintaining these systems—formatting logs, generating plans—consumes tokens better spent on iterations. The $2.9\times$ token usage for C16 supports this interpretation.

Implications for practitioners. Our results suggest investing in iteration infrastructure—better execution environments, richer feedback signals, more informative error messages—rather than architectural sophistication. The marginal engineering hour yields more return improving the feedback loop than adding agents or memory. This is practical good news: simpler systems are easier to build, debug, and maintain.

When might complexity help? Our negative results do not imply multi-agent systems or memory will never help. They may become valuable as base models improve and coordination overhead decreases, or for tasks requiring genuine parallelism. The relevant question is not whether these components can help, but whether they help now, on current models and tasks. For the component implementations we test, our evidence suggests not. Future improvements must overcome 8–14 point primary-ablation deficits, and up to 15.6 points across backbones, before complexity becomes worthwhile.

Implications for benchmarking. Differences between MLE-Bench and K-LIVE raise concerns about relying only on historical benchmarks. We advocate for live validation where possible: K-LIVE’s rolling structure costs more to evaluate but increases confidence that improvements reflect genuine capability rather than memorization.

8. Conclusion

Autonomous agents for ML engineering combine many techniques—multi-agent coordination, iterative refinement, memory systems, planning, and retrieval—but until now it was unclear which actually drive performance. We conducted approximately 4,000 controlled experiments to find out.

Our findings challenge common design assumptions. Iterative feedback contributes more to performance than architectural complexity: a single agent that can execute, observe, and revise outperforms sophisticated multi-agent systems with memory and planning. Fixed-role multi-agent coordination underperforms the iterative single-agent baseline in our evaluation, with failure traces consistent with coordination overhead and error propagation. Memory and planning provide only small gains. The most complex configuration we tested—with all components enabled—performed the worst relative to its compute cost.

We also introduced K-LIVE, a benchmark of 25 active Kaggle competitions that provides live evaluation against contemporary human baselines, minimizes contamination risk through active-competition selection, and reports contamination analysis (§3.3). Our findings replicate across MLE-Bench and K-LIVE, both K-LIVE tiers, all problem domains, and four model backbones, supporting live validation alongside historical benchmarks.

These results provide concrete guidance for practitioners: invest in iteration quality and feedback mechanisms rather than architectural sophistication. Simpler agents that iterate effectively outperform complex systems that coordinate poorly.

Impact Statement

This paper contributes to the development of autonomous ML engineering agents.

Scientific contributions. We provide the first large-scale controlled ablation of architectural components in ML engineering agents. Our 4,000-experiment study establishes that iteration dominates other factors, fixed-role multi-agent coordination underperforms, while memory and planning provide at most marginal gains. These findings challenge prevailing design assumptions and provide empirical grounding for future development.

Benchmark contribution. We introduce K-LIVE, a benchmark of 25 active Kaggle competitions providing a contamination-minimal evaluation. The two-tier structure balances freshness with longitudinal comparability. We release K-LIVE to enable rigorous evaluation of future agents.

Practical guidance. For practitioners, our results provide concrete recommendations: invest in iteration quality rather than architectural complexity. Sophisticated agent architectures risk overfitting to narrow benchmark distributions like MLE-Bench. The key insight is simpler: let the LLM improve itself through feedback from prior attempts rather than engineering elaborate coordination mechanisms.

Potential negative impacts. While this work aims to improve understanding of ML agent design, it also advances capabilities that carry potential societal risks. First, as ML engineering agents improve, they may displace workers in routine data science roles. Second, agents that autonomously build ML pipelines and optimize for a single metric may propagate or amplify biases in training data without the scrutiny a human practitioner would apply. Hence we recommend that this research should only be deployed with a human in the loop that can quality check for these ethical concerns. Lastly, the same iteration-driven capabilities that make agents effective for legitimate ML tasks could be repurposed to automate adversarial attacks, train models for disinformation, or generate malicious code at scale. We encourage the community to develop appropriate safeguards—usage monitoring, output auditing, and fairness constraints—as agent capabilities continue to advance.

References

- Chan, J. S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., Weng, L., and Madry, A. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025.
- Chopde, A., Pettiwala, F., Kirubananth, S., Botla, S. K., and Kethan, P. A. PiML: Automated machine learning workflow optimization using LLM agents. In *AutoML 2025 Methods Track*, 2025.
- DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- Guo, S., Deng, C., Wen, Y., Chen, H., Chang, Y., and Wang, J. DS-agent: Automated data science by empowering large language models with case-based reasoning. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 16813–16848. PMLR, 2024.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Hong, S., Lin, Y., Liu, B., Liu, B., Wu, B., Zhang, C., Li, D., Chen, J., Zhang, J., Wang, J., Zhang, L., Zhang, L., Yang, M., Zhuge, M., Guo, T., Zhou, T., Tao, W., Tang, R., Lu, X., Zheng, X., Liang, X., Fei, Y., Cheng, Y., Ni, Y., Gou, Z., Xu, Z., Luo, Y., and Wu, C. Data Interpreter: An LLM agent for data science. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 19796–19821. Association for Computational Linguistics, 2025.
- Huang, Q., Vora, J., Liang, P., and Leskovec, J. MLAGent-Bench: Evaluating language agents on machine learning experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 20271–20309. PMLR, 2024.
- Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Lai, Y., Li, C., Wang, Y., Zhang, T., Zhong, R., Zettlemoyer, L., Yih, W.-T., Fried, D., Wang, S., and Yu, T. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 18319–18345. PMLR, 2023.
- Li, A., Wu, C., Ge, Z., Chong, Y. H., Hou, Z., Cao, L., Ju, C., Wu, J., et al. The FM agent. *arXiv preprint arXiv:2510.26144*, 2025.
- Li, Z., Zang, Q., Ma, D., Guo, J., Zheng, T., Liu, M., Niu, X., Wang, Y., Yang, J., Liu, J., Zhong, W., Zhou, W., Huang, W., and Zhang, G. AutoKaggle: A multi-agent

framework for autonomous data science competitions.
arXiv preprint arXiv:2410.20424, 2024.

Liu, Z., Cai, Y., Zhu, X., Zheng, Y., Chen, R., Wen, Y., Wang, Y., E, W., and Chen, S. ML-Master: Towards AI-for-AI via integration of exploration and reasoning. *arXiv preprint arXiv:2506.16499*, 2025.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.

Qiang, R., Zhuang, Y., Li, Y., V K, D. S., Zhang, R., Li, C., Wong, I. S.-H., Yang, S., Liang, P., Zhang, C., and Dai, B. MLE-Dojo: Interactive environments for empowering LLM agents in machine learning engineering. *arXiv preprint arXiv:2505.07782*, 2025.

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H. H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., Narasimhan, K., Yih, W.-t., Neubig, G., and Fried, D. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., and Wang, C. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Conference on Language Modeling*, 2024.

Zhang, L., Zhang, Y., Ren, K., Li, D., and Yang, Y. ML-Copilot: Unleashing the power of large language models in solving machine learning tasks. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2931–2959. Association for Computational Linguistics, 2024.

A. K-LIVE Benchmark Specification

A.1. Kaggle Medal Thresholds

Table 12 shows Kaggle’s official medal thresholds, which vary by competition size. These thresholds are used by MLE-Bench to compute medal rates.

Table 12. Kaggle medal thresholds by competition size.

Medal	0–99 teams	100–249 teams	250–999 teams	1000+ teams
Bronze	Top 40%	Top 40%	Top 100	Top 10%
Silver	Top 20%	Top 20%	Top 50	Top 5%
Gold	Top 10%	Top 10	Top 10 + 0.2%	Top 10 + 0.2%

Because threshold requirements vary with competition size, medal rates are not directly comparable across competitions of different scales. A bronze medal in a 50-team competition (top 40%) requires less competitive performance than bronze in a 5,000-team competition (top 10%). This inconsistency motivates our use of percentile rank in K-LIVE, which provides uniform granularity regardless of competition size.

A.2. Two-Tier Structure

K-LIVE organizes 25 competitions into two tiers. **Tier 1** contains 13 active competitions launched recently enough that their data is unlikely to appear in LLM training corpora before the relevant model knowledge cutoffs. Their leaderboards reflect contemporary human performance using modern techniques. These rotate as competitions conclude. **Tier 2** contains 12 perpetual competitions providing stable baselines for longitudinal comparison.

This structure enables differentiated evaluation: Tier 1 measures problem-solving with minimal contamination risk; Tier 2 measures reliability on standard tasks. Consistent performance across tiers indicates robust capability.

A.3. Tier 1: Active Competitions

Table 13 lists the 13 Tier 1 competitions in K-LIVE-2026-01. Selection criteria: (1) active competitions from the January 2026 snapshot, with exceptions for high-quality unsolved problems; (2) ≥ 150 participating teams; (3) tractable within 24 hours on an A100; (4) domain diversity.

Table 13. Tier 1: Active competitions in K-LIVE-2026-01. Competition IDs are Kaggle slugs. Start dates are included to support per-model cutoff evaluation (Section 3.3).

#	Competition ID	Start Date	Deadline	Data Type	Teams	Difficulty
1	ai-mathematical-olympiad-progress-prize-3	Nov 2025	Apr 2026	Text (Math)	3,000	Very High
2	hull-tactical-market-prediction	Sep 2025	Jun 2026	Tabular (Financial)	3,800	Very High
3	recodai-luc-scientific-image-forgery-detection	Oct 2025	Apr 2026	Image (Forensics)	1,600	High
4	llm-agentic-legal-information-retrieval	Jan 2026	May 2026	Text (Legal)	150	High
5	deep-past-initiative-machine-translation	Dec 2025	Mar 2026	Multimodal	900	High
6	stanford-rna-3d-folding-2	Jan 2026	Mar 2026	Sequence (Bio)	1,200	Very High
7	vesuvius-challenge-surface-detection [†]	Nov 2025	Feb 2026	Volumetric (3D)	1,500	High
8	med-gemma-impact-challenge	Jan 2026	Feb 2026	Multimodal (Clinical)	600	Moderate
9	wbc-bench-2026	Nov 2025	May 2026	Image (Microscopy)	300	High
10	urban-flood-modelling	Jan 2026	Mar 2026	Tabular/Time Series	500	Moderate
11	playground-series-s6e1	Dec 2025	Jan 2026	Tabular (Synthetic)	2,500	Moderate
12	bidding-predictions-for-construction	Jan 2026	Mar 2026	Tabular/Geospatial	400	Moderate
13	nfl-big-data-bowl-2026-prediction [†]	Sep 2025	Jan 2026	Tracking (Sports)	800	High

[†]Included despite shorter deadline due to exceptional quality and unsolved nature.

A.4. Tier 2: Perpetual Competitions

Table 14 lists the 12 Tier 2 competitions. These are perpetual “Getting Started” or community competitions that remain open indefinitely. They stay constant across K-LIVE versions, enabling longitudinal comparison.

Table 14. Tier 2: Perpetual competitions in K-LIVE-2026-01. Competition IDs are Kaggle slugs. Team counts approximate as of January 2026.

#	Competition ID	Data Type	Teams	Size	Difficulty
14	spaceship-titanic	Tabular (Classification)	2,400	2 MB	Moderate
15	store-sales-time-series-forecasting	Time Series (Retail)	900	20 MB	Moderate
16	llm-classification-finetuning	Text (Classification)	300	Medium	High
17	digit-recognizer	Image (Digits)	1,600	15 MB	Low
18	house-prices-advanced-regression-techniques	Tabular (Regression)	4,300	<1 MB	Low
19	nlp-getting-started	Text (Classification)	700	1 MB	Low
20	connectx	Simulation (RL)	250	N/A	Low
21	tpu-getting-started	Image (Fine-grained)	250	Large	Moderate
22	contradictory-my-dear-watson	Text (NLI)	180	Small	Low
23	gan-getting-started	Image (GAN)	130	Large	Moderate
24	home-data-for-ml-course	Tabular (Regression)	4,500	<1 MB	Low
25	landmark-recognition-2021	Image (Retrieval)	750	1+ GB	Moderate

A.5. Domain Distribution

Table 15 summarizes fine-grained domain coverage. Unlike MLE-Bench and MLE-Dojo (primarily tabular/vision/NLP), K-LIVE includes biological sequences, medical imaging, 3D volumetric data, and sports analytics.

Note: For performance analysis in Table 16 we use coarser categories: Vision includes generative AI tasks; Tabular includes medical and clinical data; NLP is unchanged; Time series includes RL/simulation and sports analytics; Other includes biological sequences, 3D volumetric, and multimodal tasks. Under this mapping, K-LIVE contributes 5 Vision, 8 Tabular, 5 NLP, 4 Time series, and 3 Other competitions.

Table 15. Domain distribution across K-LIVE-2026-01.

Domain	Total	Tier 1	Tier 2
Tabular/Structured	6	3	3
Computer Vision	4	1	3
NLP	5	2	3
Time Series	2	1	1
Biological Sequences	1	1	0
Medical/Clinical	2	2	0
3D/Volumetric	1	1	0
Multimodal	1	1	0
RL/Simulation	1	0	1
Generative AI	1	0	1
Sports Analytics	1	1	0
Total	25	13	12

Table 16. K-LIVE performance by domain. Iteration dominance holds across all domains; multi-agent penalty is largest on complex tasks.

Domain	n	C1	C2	C3
Tabular	8	86.4	52.1	79.8
NLP	5	82.3	48.7	72.1
Vision	5	76.8	41.2	66.4
Time series	4	80.1	46.3	71.8
Other	3	75.1	42.8	67.3

A.6. Versioning

Tier 1 competitions rotate as deadlines pass. We publish versioned snapshots (e.g., K-LIVE-2026-01) specifying the exact competition set and leaderboard state at freeze time. Tier 2 remains constant across versions unless severe leakage is discovered.

A.7. Evaluation Protocol

Performance is measured as percentile rank: for rank R among N teams, $\text{Percentile} = 100 \times (N - R + 1)/N$. We report mean percentile across all competitions, with separate Tier 1, Tier 2, and domain-specific breakdowns.

All submissions use the official Kaggle API. Each configuration receives 24 wall-clock hours per competition. Hardware node: A100 (80GB), 512GB RAM, 2TB SSD.

B. Agent Prompts

This appendix provides the core system prompts used in our experiments. We designed prompts following three principles: (1) *role clarity*—each agent has a well-defined responsibility; (2) *structured output*—agents produce machine-parseable artifacts for downstream processing; and (3) *minimal domain assumptions*—prompts specify methodology, not domain-specific techniques. The same prompts were used across all 100 competitions without modification except where explicitly varied in ablations.

B.1. Single-Agent Baseline

The single-agent configuration uses a unified prompt combining execution, iteration, and submission handling.

Single-Agent System Prompt

You are an ML engineering agent. Your task is to analyze a competition, develop a solution, and produce a submission that maximizes leaderboard performance.

Environment

- Data directory: `/data/` (contains train, test, and sample submission)
- Workspace: `/workspace/` (your working directory)
- Time budget: 24 hours wall-clock
- Hardware: GPU available via PyTorch/TensorFlow

Workflow

1. **Understand:** Read the problem description and examine the data
2. **Baseline:** Implement a simple working solution first
3. **Iterate:** Improve based on validation feedback
4. **Submit:** Generate predictions in the required format

Validation Requirements

- Use cross-validation ($k \geq 5$) matching the evaluation metric
- Report mean and standard deviation across folds
- Never use test data for any training decisions

After Each Experiment

Call `LogExperiment` with:

- `name`: Descriptive experiment identifier
- `cv_score`: Cross-validation score (verified from output)
- `approach`: Brief description of methodology
- `submission_path`: Path to generated submission file

Iteration Protocol

When feedback is available from previous iterations:

1. Review execution output (errors, warnings, metrics)
2. Identify the primary bottleneck (bugs, model capacity, features)
3. Implement targeted improvements
4. Verify improvement via cross-validation before proceeding

B.2. Multi-Agent: Explorer

The Explorer agent analyzes the competition and produces a strategy document for downstream agents.

Explorer System Prompt

You are an exploration specialist. Your task is to analyze a machine learning competition and produce a strategy document.

Your Output

Write `strategy.json` containing:

- `problem_type`: Classification, regression, ranking, etc.
- `evaluation_metric`: The competition's scoring function
- `data_characteristics`: Key properties (size, features, class balance)
- `recommended_approaches`: Ordered list of techniques to try
- `potential_challenges`: Anticipated difficulties

Analysis Process

1. Read the problem description thoroughly
2. Examine data shapes, types, and distributions
3. Identify the evaluation metric and its implications
4. Note any special constraints (submission format, runtime limits)
5. Formulate approach recommendations based on problem structure

Constraints

- Time limit: 10 minutes
- Do not implement solutions; focus on analysis
- Base recommendations on data properties, not assumptions

B.3. Multi-Agent: Builder

The Builder agent implements solutions based on the Explorer's strategy.

Builder System Prompt

You are an implementation specialist. Your task is to build ML solutions based on a provided strategy document.

Input

Read `strategy.json` from the Explorer to understand:

- Problem type and evaluation metric
- Data characteristics
- Recommended approaches (implement in order)

Implementation Standards

- Implement cross-validation matching the evaluation metric
- Handle edge cases (missing values, class imbalance)
- Log all experiments with reproducible configurations
- Generate submission files matching the required format

Workflow

1. Start with the first recommended approach
2. Implement a working baseline before optimizing
3. Log results after each experiment
4. If time permits, proceed to subsequent approaches

Communication

After each experiment, update `results.json` with:

- Experiment name and approach description
- Cross-validation score with standard deviation
- Path to submission file
- Any issues encountered

B.4. Multi-Agent: Evaluator

The Evaluator agent reviews implementations for correctness before submission.

Evaluator System Prompt

You are a code review specialist. Your task is to verify that ML experiments are correctly implemented before their results are trusted.

Review Checklist

Validation Integrity

- Is cross-validation implemented correctly?
- Does the CV scheme match the problem structure?
- Are scores computed on held-out folds only?

Data Leakage

- Is preprocessing fitted on training data only?
- Are features computed within CV folds?
- Is test data isolated from all training decisions?

Submission Format

- Does the submission match the sample format?
- Are all required rows and columns present?
- Are predictions in the correct data type?

Output

Write `review.json` containing:

- `verdict`: PASS, CONCERNS, or FAIL
- `issues`: List of identified problems
- `recommendations`: Suggested fixes if applicable

B.5. Planning Template

When planning is enabled, agents generate a structured plan before implementation.

Planning Template

Before writing code, produce a plan with the following structure:

Problem Analysis

- Problem type (classification, regression, etc.)
- Evaluation metric and optimization direction
- Key data characteristics

Approach

- Selected modeling approach with justification
- Preprocessing steps required
- Expected challenges and mitigations

Execution Steps

1. Data loading and initial exploration
2. Preprocessing pipeline implementation
3. Model training with cross-validation
4. Submission generation

Success Criteria

- Target validation score (based on baseline)
- Failure indicators to watch for

Output the plan, then proceed with implementation.

B.6. Memory System

When memory is enabled, agents maintain a structured log of past experiments that persists across iterations.

Memory System Instructions

A memory system is available to track your experiment history.

Logging Experiments

After each experiment, the system records:

- Experiment identifier and timestamp
- Approach description and hyperparameters
- Cross-validation score and standard deviation
- Any errors or warnings encountered

Querying Memory

Before starting a new experiment:

1. Review past experiments to avoid repetition
2. Identify approaches that have not been tried
3. Note which techniques improved or degraded performance
4. Build on successful approaches rather than starting fresh

Memory Contents

The file `experiment_log.json` contains all past experiments with their configurations and results. Reference this before planning new experiments.

B.7. Prompt Design Rationale

Our prompts were designed to isolate component effects while maintaining ecological validity:

Consistency. The same base prompt was used across all competitions. No domain-specific hints were provided, ensuring findings generalize across problem types.

Minimality. Prompts specify what to do (validate properly, log experiments) but not how to solve specific problems. This

prevents prompt-level overfitting to particular competition types.

Structured outputs. Multi-agent prompts require JSON artifacts for inter-agent communication. This standardizes handoffs and makes coordination failures detectable in our analysis.

Reproducibility. All prompts are deterministic text with no dynamic content except the planning and routing components being ablated. The complete prompt text for each configuration is available in our code repository.

C. Implementation Details

This appendix provides implementation details necessary for reproducing our experiments.

C.1. Component Specifications

Iteration. Non-iterative agents (1 round) produce a solution in a single pass without observing execution results. Iterative agents (10 rounds) observe stdout, stderr, validation metrics, and public leaderboard scores, then revise their approach. A typical trajectory: round 0 establishes a baseline; rounds 1–2 fix bugs and tune hyperparameters; rounds 3+ explore alternatives when progress stalls.

Multi-agent. The single-agent baseline maintains one conversation thread throughout. The multi-agent system uses three agents: (1) Explorer analyzes data distributions, missing values, and correlations; (2) Builder writes training code based on Explorer’s findings; (3) Evaluator reviews outputs for errors like data leakage or format issues. Agents communicate via structured JSON messages.

Memory. Without memory, agents rely on the context window alone. With memory, agents maintain a structured `experiment_log.json` file containing the experiment ID, approach description, key code changes, validation results, errors, and assessment for each completed experiment. Before planning the next experiment, the agent consults this log to identify prior attempts, avoid repeated failures, and build on approaches that showed evidence of improvement.

Planning. Reactive agents decide what to do based on immediate observations. Planning agents first generate explicit strategy: problem type, likely challenges, promising approaches, and experiment sequence. Plans may be revised based on feedback when iteration is also enabled.

Retrieval. Without retrieval, agents use only parametric knowledge. With retrieval, agents can query public Kaggle resources, such as notebooks and datasets. We ablate retrieval (C6) on MLE-Bench only. On K-LIVE Tier 1, public materials are sparse and incomplete; on K-LIVE Tier 2, public notebooks exist, making retrieval qualitatively different from Tier 1. We therefore do not report a K-LIVE retrieval ablation.

C.2. Infrastructure

All experiments were conducted on a dedicated cluster. Each agent run executes in an isolated Docker container with no network access except to the Kaggle API endpoint.

Table 17. Hardware configuration per experimental run.

Component	Specification
GPU	NVIDIA A100 (80GB)
CPU	64-core AMD EPYC 7763
RAM	512 GB DDR4
Storage	2 TB NVMe SSD

C.3. Model Configuration

The primary ablation configurations use DeepSeek-V3.2 as the base language model. The cross-model backbone check reruns C1–C5 and C16 with Kimi K2.5, Claude Sonnet 4.6, and Gemini 3 Flash Preview using the same agent prompts, tools, and evaluation protocol.

Table 18. Language model inference parameters.

Parameter	Value
Primary model	DeepSeek-V3.2
Cross-model backbones	Kimi K2.5, Claude Sonnet 4.6, Gemini 3 Flash Preview
Context length	128,000 tokens
Temperature	0.7
Top-p	0.95
Max output tokens	4,096 per turn
Timeout per call	120 seconds

C.4. Kaggle API Integration

Agents submit predictions and, when retrieval is enabled, access public Kaggle resources through the official Kaggle API. Endpoint access is configuration-dependent so that the retrieval ablation changes only the availability of external public resources.

Always allowed endpoints:

- `competitions/data/download`: Download competition data
- `competitions/submissions/submit`: Submit predictions
- `competitions/submissions/list`: Check submission status

Retrieval-enabled endpoints (available only when retrieval is on):

- `kernels/list`: List public notebooks
- `kernels/pull`: Download notebook code
- `datasets/list`: List public datasets

When retrieval is disabled, the retrieval-enabled endpoints are blocked. This prevents agents in the no-retrieval condition from accessing public notebooks, notebook code, or auxiliary public datasets beyond the official competition data.

C.5. Run Management

Each experimental run follows this lifecycle:

1. **Initialization**: Spawn Docker container, mount competition data, initialize agent with configuration-specific prompts.
2. **Execution**: Agent runs for up to 24 wall-clock hours. All actions, tool outputs, and LLM responses are logged.
3. **Termination**: Run ends when (a) agent declares completion, (b) 24-hour limit reached, or (c) unrecoverable error occurs.
4. **Scoring**: Final submission score retrieved from Kaggle. If no valid submission exists, run is marked as failed.

Reported score means are computed over successful runs only: each valid submission from a configuration, competition, and replicate contributes one score. Failed runs (no valid submission) are excluded from percentile calculations but included in failure rate analysis.

D. Extended Results

This appendix provides complete results for all 16 configurations in our ablation study.

Table 19. Ablation study design: 16 configurations across 4 studies.

ID	Configuration	Iter	Multi	Mem	Plan	Ret	Purpose
<i>Study 1: Single-Component Ablations</i>							
C1	Baseline	10	–	–	–	✓	Reference
C2	No iteration	1	–	–	–	✓	Iteration effect
C3	+ Multi-agent	10	✓	–	–	✓	Multi-agent effect
C4	+ Memory	10	–	✓	–	✓	Memory effect
C5	+ Planning	10	–	–	✓	✓	Planning effect
C6	No retrieval	10	–	–	–	–	Retrieval effect
<i>Study 2: Iteration Interactions</i>							
C7	Multi-agent + 3 iter	3	✓	–	–	✓	Multi-agent at low iter?
C8	Multi-agent + 5 iter	5	✓	–	–	✓	Multi-agent at mid iter?
C9	Memory + 5 iter	5	–	✓	–	✓	Memory reduces iter need?
C10	Planning + 5 iter	5	–	–	✓	✓	Planning reduces iter need?
C11	Planning + 3 iter	3	–	–	✓	✓	Planning at low iter?
<i>Study 3: Component Combinations</i>							
C12	Multi-agent + Memory	10	✓	✓	–	✓	Memory rescues multi?
C13	Multi-agent + Planning	10	✓	–	✓	✓	Planning rescues multi?
C14	Memory + Planning	10	–	✓	✓	✓	Deliberation synergy?
C15	Multi + Mem + Plan	10	✓	✓	✓	✓	Triple combination
<i>Study 4: Maximum Complexity</i>							
C16	All components	10	✓	✓	✓	✓	Full system

D.1. Experimental Design

We tested 16 configurations organized into four studies. Table 19 specifies each configuration.

Run counts:

- Study 1: 6 configs $\times$ 100 competitions $\times$ 3 seeds = 1,800 runs (C6 on MLE-Bench only: –75 runs)
- Study 2: 5 configs $\times$ 100 $\times$ 3 = 1,500 runs
- Study 3: 4 configs $\times$ 100 $\times$ 3 = 1,200 runs
- Study 4: 1 config $\times$ 100 $\times$ 3 = 300 runs
- **Total: 4,725 runs attempted; 3,973 successful (15.9% failure rate)**

Note on metrics: MLE-Bench scores represent medal rates (% of competitions achieving bronze or better). K-LIVE scores represent percentile rank on the public leaderboard. These metrics are not directly comparable but both measure competitive performance.

D.2. Study 1: Single-Component Effects

Table 20 shows complete results for all single-component ablations.

Key findings:

- **Iteration dominates:** Removing iteration causes a 33.7-point drop in MLE-Bench medal rate and 34.1-point drop in K-LIVE percentile. This is by far the largest effect.
- **Fixed-role multi-agent coordination underperforms:** The fixed-role pipeline decreases performance by 8.3 points on MLE-Bench and 8.5 points on K-LIVE.
- **Memory and planning are marginal:** Both show only small gains.
- **Retrieval helps on historical benchmarks:** 11.2-point drop without retrieval (on MLE-Bench).

Table 20. Study 1: Single-component ablation results. MLE-Bench = medal rate (%). K-LIVE = percentile rank.

Config	MLE-Bench (%)		K-LIVE (ptl)		Δ vs C1	
	Mean	Std	Mean	Std	MLE	K-LIVE
C1: Baseline	52.4	4.1	81.3	6.2	—	—
C2: No iter	18.7	5.8	47.2	9.4	−33.7	−34.1
C3: Multi	44.1	4.9	72.8	7.1	−8.3	−8.5
C4: Memory	53.8	4.0	82.1	6.0	+1.4	+0.8
C5: Planning	53.1	4.2	81.9	6.1	+0.7	+0.6
C6: No retr	41.2	4.7	—	—	−11.2	—

Table 21. Cross-model K-LIVE results for key configurations. Scores are percentile ranks; Δ is C3 − C1.

Backbone	C1	C2	C3	Δ	C4	C5	C16
DeepSeek-V3.2	81.3	47.2	72.8	−8.5	82.1	81.9	67.4
Kimi K2.5	87.1	61.0	72.1	−15.0	84.9	88.6	69.5
Claude Sonnet 4.6	88.9	62.6	73.3	−15.6	85.8	89.7	69.8
Gemini 3 Flash Preview	86.3	61.2	71.4	−14.9	85.3	88.2	68.6

D.3. Cross-Model Backbone Check

To test whether the single-component conclusions depend on the DeepSeek-V3.2 backbone, we reran the six highest-signal configurations on three additional frontier models as shown in Table 21. These runs reuse the same K-LIVE competition set, prompts, tools, and 24-hour budget.

The qualitative ranking of components is stable across backbones: iteration remains dominant, memory and planning remain small or neutral, and the all-components configuration remains below the baseline. The multi-agent result is especially consistent: stronger backbones raise the single-agent baseline but do not raise the fixed-role multi-agent configuration, which stays near 71–73 percentile across all four models.

Table 22. No component compensates for iteration. Effect column shows delta versus single-agent baseline at matched iteration count.

Configuration	Iter	MLE (%)	Baseline	Effect
C7: Multi-agent	3	29.1	38.4	−9.3
C8: Multi-agent	5	36.2	45.6	−9.4
C9: Memory	5	44.8	45.6	−0.8
C10: Planning	5	44.2	45.6	−1.4
C11: Planning	3	36.8	38.4	−1.6

D.4. Study 2: Iteration Interactions

This study tests whether components interact with iteration count as an extension of Table 22. The key question: *do multi-agent, memory, or planning reduce the need for iteration?*

First, we establish baseline performance at different iteration counts:

Now we test components at reduced iteration counts:

Key findings:

- **Fixed-role coordination does not compensate for iteration:** The multi-agent penalty is roughly constant across iteration counts: −8.3 on MLE-Bench and −8.5 on K-LIVE at 10 iterations, −9.3 and −9.0 at 3 iterations, and −9.4 and −9.0 at 5 iterations.
- **Memory doesn’t reduce iteration needs:** Memory at 5 iter (44.8%) performs similarly to baseline at 5 iter (45.6%).

Table 23. Baseline (single-agent) performance by iteration count.

Iterations	MLE-Bench (%)	K-LIVE (pctl)	Δ MLE vs 1 iter	Marginal MLE/iter
1	18.7	47.2	—	—
3	38.4	65.8	+19.7	+9.85
5	45.6	74.1	+26.9	+3.6
7	49.8	78.9	+31.1	+2.1
10	52.4	81.3	+33.7	+0.87

Table 24. Study 2: Iteration interaction results. Effects are deltas versus the single-agent baseline at matched iteration count.

Config	Scores		Effect vs Baseline@iter	
	MLE (%)	K-LIVE (pctl)	MLE	K-LIVE
C7: Multi + 3 iter	29.1	56.8	−9.3	−9.0
C8: Multi + 5 iter	36.2	65.1	−9.4	−9.0
C9: Memory + 5 iter	44.8	74.6	−0.8	+0.5
C10: Planning + 5 iter	44.2	73.9	−1.4	−0.2
C11: Planning + 3 iter	36.8	66.4	−1.6	+0.6

- **Planning doesn’t reduce iteration needs:** Planning at 5 iter (44.2%) and 3 iter (36.8%) shows no clear improvement over baseline at same iteration counts.

D.5. Study 3: Component Combinations

This study tests whether component pairs synergize.

Table 25. Study 3: Interaction effects by metric. Expected effects are sums of the corresponding single-component deltas; interactions are observed minus expected.

Metric	Config	Expected	Observed	Interaction	Finding
MLE-Bench	C12: Multi + Memory	−6.9	−7.2	−0.3	Additive
MLE-Bench	C13: Multi + Planning	−7.6	−7.6	0.0	Additive
MLE-Bench	C14: Memory + Planning	+2.1	+0.8	−1.3	Additive
MLE-Bench	C15: Multi + Mem + Plan	−6.2	−9.3	−3.1	Negative synergy
K-LIVE	C12: Multi + Memory	−7.7	−7.9	−0.2	Additive
K-LIVE	C13: Multi + Planning	−7.9	−8.4	−0.5	Additive
K-LIVE	C14: Memory + Planning	+1.4	+0.4	−1.0	Additive
K-LIVE	C15: Multi + Mem + Plan	−7.1	−10.1	−3.0	Negative synergy

Interaction analysis: We test whether observed effects differ from the sum of individual effects.

Key findings:

- **Memory does not rescue multi-agent (C12):** The multi-agent penalty persists (C12 vs C3: −7.2 vs −8.3 on MLE-Bench; −7.9 vs −8.5 on K-LIVE).
- **Planning does not rescue multi-agent (C13):** Similar pattern (−7.6 on MLE-Bench; −8.4 on K-LIVE).
- **Memory + Planning is neutral (C14):** No synergy, but no harm either.
- **Triple combination shows negative synergy (C15):** Performance is 3.1 points worse than expected from individual effects on MLE-Bench and 3.0 points worse on K-LIVE.

D.6. Study 4: Maximum Complexity

Key finding: The most complex configuration (C16) performs 13.7 points worse than baseline on MLE-Bench and 13.9 points worse on K-LIVE, while using $2.9\times$ as many tokens. Combining the tested components produces the worst performance of any configuration in terms of cost-performance.

Table 26. Study 4: All-components configuration.

	MLE (%)	Δ	K-LIVE (pctl)	Δ	Tokens	Cost
C16: All	38.7	-13.7	67.4	-13.9	10.2M	2.9×
C1: Baseline	52.4	—	81.3	—	3.5M	1.0×

D.7. Complete Results Table

Table 27 shows all 16 configurations with both metrics.

Table 27. Complete results for all 16 configurations.

Config	MLE-Bench (%)	K-LIVE (pctl)	Δ MLE	Δ K-LIVE
C1: Baseline	52.4	81.3	—	—
C2: No iteration	18.7	47.2	-33.7	-34.1
C3: Multi-agent	44.1	72.8	-8.3	-8.5
C4: Memory	53.8	82.1	+1.4	+0.8
C5: Planning	53.1	81.9	+0.7	+0.6
C6: No retrieval	41.2	—	-11.2	—
C7: Multi + 3 iter	29.1	56.8	-23.3	-24.5
C8: Multi + 5 iter	36.2	65.1	-16.2	-16.2
C9: Memory + 5 iter	44.8	74.6	-7.6	-6.7
C10: Planning + 5 iter	44.2	73.9	-8.2	-7.4
C11: Planning + 3 iter	36.8	66.4	-15.6	-14.9
C12: Multi + Memory	45.2	73.4	-7.2	-7.9
C13: Multi + Planning	44.8	72.9	-7.6	-8.4
C14: Memory + Planning	53.2	81.7	+0.8	+0.4
C15: Multi + Mem + Plan	43.1	71.2	-9.3	-10.1
C16: All components	38.7	67.4	-13.7	-13.9

D.8. Failure Analysis

This section provides detailed failure analysis across all 16 configurations. A run is considered a “failure” if it produces no valid submission by the 24-hour deadline.

Key observations:

- **Multi-agent configurations fail 1.7–2.3× more often** than the baseline. C7 (multi-agent + 3 iterations) has the highest failure rate at 23.7%.
- **No-iteration (C2) has high failure rate (22.7%)** because agents cannot recover from initial errors.
- **Memory and planning do not increase failures**—their rates (9.7% and 10.7%) are comparable to baseline (10.3%).
- **Failure modes differ by configuration type**: Single-agent failures are dominated by code execution errors (capability limits); multi-agent failures shift toward coordination errors and timeouts.

The shift in failure modes helps explain why the tested fixed-role pipeline underperforms: it trades capability-limited failures for coordination-limited failures. Since coordination failures occur more frequently in these runs, net performance decreases.

D.9. Cost Analysis

Table 30 shows token consumption for each configuration. Costs are computed using DeepSeek-V3.2 API pricing (\$0.14/1M input tokens, \$0.28/1M output tokens) and normalized relative to baseline.

Key observations:

- **Multi-agent doubles token usage**: C3 uses 2.1× baseline tokens due to inter-agent communication overhead.

Table 28. Failure rates by configuration.

Configuration	Runs	Failures	Rate	Pattern
C1: Baseline	300	31	10.3%	Reference
C2: No iteration	300	68	22.7%	High: no recovery
C3: Multi-agent	300	54	18.0%	High: coordination
C4: Memory	300	29	9.7%	Normal
C5: Planning	300	32	10.7%	Normal
C6: No retrieval	225	27	12.0%	Slightly elevated
C7: Multi + 3 iter	300	71	23.7%	Highest
C8: Multi + 5 iter	300	62	20.7%	High
C9: Memory + 5 iter	300	38	12.7%	Normal
C10: Planning + 5 iter	300	39	13.0%	Normal
C11: Planning + 3 iter	300	48	16.0%	Elevated
C12: Multi + Memory	300	52	17.3%	High
C13: Multi + Planning	300	53	17.7%	High
C14: Memory + Planning	300	30	10.0%	Normal
C15: Multi + Mem + Plan	300	56	18.7%	High
C16: All	300	62	20.7%	High
Total	4,725	752	15.9%	

Table 29. Failure mode breakdown.

Mode	Count	%	Primary cause
Code execution error	242	32.2%	Syntax, imports, shapes
Timeout (24h exceeded)	212	28.2%	Complex models, large data
Out of memory	110	14.6%	Vision models, ensembles
Invalid submission format	94	12.5%	Missing columns, wrong dtype
Infrastructure error	55	7.3%	GPU failures, network
Agent stuck (no progress)	39	5.2%	Loops, repeated failures
Total	752	100%	

- **Memory and planning add modest overhead:** $1.2\times$ baseline each, primarily from structured logging and plan generation.
- **All-components (C16) costs $2.9\times$ baseline** while performing 13.7 points worse, the worst cost-performance tradeoff.
- **Reduced iterations save cost:** C2 (no iteration) uses only $0.3\times$ baseline tokens, but performance drops 33.7 points.

Cost-effectiveness analysis: The baseline (C1) provides the strongest cost-performance tradeoff among the full-iteration configurations we evaluate. Adding iterations improves performance but with diminishing returns: going from 3 to 10 iterations costs roughly $3\times$ more tokens for 14 additional MLE-Bench points. Multi-agent configurations are dominated by single-agent configurations at comparable token budgets: they consume similar or greater tokens while performing worse.

D.10. Extended Iso-Cost Comparison

To rigorously evaluate whether the multi-agent penalty is simply an artifact of fixed iteration counts rather than total token budgets, we provide a comprehensive iso-cost comparison. Table 31 groups configurations by their total token consumption to illustrate performance at comparable cost tiers.

As shown, multi-agent configurations consistently underperform single-agent configurations even when granted equivalent or strictly larger token budgets. For instance, the baseline (C1) achieves 81.3 percentile on K-LIVE using 3.52M tokens. The closest multi-agent configuration in cost is C7 (2.90M tokens), which achieves only 56.8 percentile.

Furthermore, even when the multi-agent pipeline is given more than double the token budget (C3, 7.36M tokens), it still underperforms the cheaper single-agent baseline by 8.5 percentile points. This extended breakdown confirms that the fixed-role multi-agent pipeline tested in our evaluation is strictly dominated on a cost-performance basis, and its poor

Table 30. Token usage and relative cost by configuration.

Config	Input (M)	Output (M)	Total (M)	Cost
C1: Baseline	2.84	0.68	3.52	1.0×
C2: No iteration	0.89	0.21	1.10	0.3×
C3: Multi-agent	5.94	1.42	7.36	2.1×
C4: Memory	3.41	0.82	4.23	1.2×
C5: Planning	3.19	0.89	4.08	1.2×
C7: Multi + 3 iter	2.34	0.56	2.90	0.8×
C8: Multi + 5 iter	3.78	0.91	4.69	1.3×
C9: Memory + 5 iter	2.14	0.51	2.65	0.8×
C10: Planning + 5 iter	2.01	0.56	2.57	0.7×
C11: Planning + 3 iter	1.28	0.36	1.64	0.5×
C12: Multi + Memory	6.51	1.56	8.07	2.3×
C13: Multi + Planning	6.28	1.74	8.02	2.3×
C14: Memory + Planning	3.62	1.01	4.63	1.3×
C15: Multi + Mem + Plan	7.24	1.89	9.13	2.6×
C16: All	8.34	1.82	10.16	2.9×

performance cannot be attributed to token starvation.

Table 31. Extended iso-cost comparison. Configurations are grouped by token budget tiers to demonstrate cost-performance tradeoffs. Multi-agent configurations consistently underperform single-agent setups across all cost tiers.

Budget Tier	Configuration	Tokens (M)	Cost Multiplier	MLE (%)	K-LIVE (pctl)
<i>Low Budget</i>	C2: No iteration (Single)	1.10	0.3×	18.7	47.2
	C11: Planning + 3 iter (Single)	1.64	0.5×	36.8	66.4
<i>Base Budget (~3M)</i>	C7: Multi + 3 iter	2.90	0.8×	29.1	56.8
	C1: Baseline (Single)	3.52	1.0×	52.4	81.3
	C5: Planning (Single)	4.08	1.2×	53.1	81.9
<i>Medium Budget (~5M)</i>	C14: Mem + Plan (Single)	4.63	1.3×	53.2	81.7
	C8: Multi + 5 iter	4.69	1.3×	36.2	65.1
<i>High Budget (~7M+)</i>	C3: Multi + 10 iter	7.36	2.1×	44.1	72.8
	C16: All components	10.16	2.9×	38.7	67.4

D.11. Dynamic Baseline Analysis

We tracked agent rankings on K-LIVE over 30 days after submission.

Rankings dropped 4.1 points over 30 days as human competitors uploaded improved solutions.

D.12. Summary of Findings

Our 16-configuration ablation study yields five main findings:

1. **Iteration dominates.** Removing iteration causes the largest performance drop (-33.7 points on MLE-Bench, -34.1 on K-LIVE). No other component comes close.
2. **Fixed-role multi-agent coordination underperforms.** Adding fixed-role multi-agent coordination decreases performance by 8.3 points on MLE-Bench and 8.5 points on K-LIVE. The penalty grows at lower iteration counts.
3. **Memory and planning have at most marginal effects.** Both components show small gains ($+1.4$ and $+0.7$ points respectively). They do not reduce iteration needs.
4. **The tested components do not rescue fixed-role coordination.** Adding memory or planning to the multi-agent pipeline does not recover the lost performance. The triple combination (C15) shows negative synergy: 3.1 points worse than expected on MLE-Bench and 3.0 points worse on K-LIVE.

Table 32. Ranking decay over time (K-LIVE, baseline configuration).

Time	Mean Percentile	Change	Active Competitors
Day 0	85.4	—	1,247
Day 7	83.6	−1.8	1,412
Day 14	82.5	−2.9	1,589
Day 30	81.3	−4.1	1,823

5. **Combining the tested components hurts.** The all-components configuration (C16) performs 13.7 points worse than baseline on MLE-Bench while costing $2.9\times$ as much. In our evaluation, adding the tested component implementations does not improve over the iterative single-agent baseline.

Practical recommendation: In the setting we evaluate, prioritize single-agent iterative refinement before adding fixed-role coordination, structured memory, or explicit planning. Fixed-role multi-agent coordination adds substantial cost and underperforms the baseline; memory and planning provide at most marginal gains.

E. Example Agent Trajectories

This appendix presents annotated trajectories from our experiments, illustrating both successful patterns and failure modes. We selected examples that demonstrate key findings from the main paper: the importance of iteration for error recovery, the coordination challenges in multi-agent systems, and the value of memory for avoiding repeated mistakes. Competition-specific details are anonymized; the patterns generalize across our benchmark.

E.1. Successful Single-Agent Trajectory

The following trajectory demonstrates a successful single-agent run achieving the 73rd percentile in 4 iterations. This example illustrates why iteration is the most impactful component: early iterations fix critical bugs, while later iterations optimize model performance.

ITERATION 1: INITIAL IMPLEMENTATION

Agent Output (Iteration 1)

Reasoning: Starting with data exploration to understand the prediction task.

> execute.code

```
import pandas as pd
train = pd.read_csv('/data/train.csv')
test = pd.read_csv('/data/test.csv')
print(f"Train: {train.shape}, Test: {test.shape}")
print(f"Target distribution:\n{train['target'].value_counts(normalize=True)}")
print(f"Missing values: {train.isnull().sum().sum()}")
```

Output:

```
Train: (8693, 14), Test: (4277, 13)
Target distribution:
0    0.497
1    0.503
Missing values: 1842
```

Analysis: Balanced binary classification with 13 features. Significant missing values require handling. Will implement baseline with gradient boosting and simple imputation.

The agent correctly identifies the problem type and key data characteristics before implementing a solution.

ITERATION 2: BUG FIX

Agent Output (Iteration 2)

Previous iteration error:

ValueError: Input contains NaN. LightGBM cannot handle missing values in categorical features encoded as integers.

Reasoning: The error indicates my categorical encoding produced NaN values. Need to handle missing values before encoding.

> execute_code

```
# Fix: Impute before encoding
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='most_frequent')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)
```

Result: Code executes successfully. Proceeding with model training.

This illustrates the error correction pattern that dominates early iterations. Without feedback, this run would have failed completely.

ITERATION 3: MODEL TRAINING

Agent Output (Iteration 3)

Reasoning: With preprocessing fixed, implementing cross-validated gradient boosting.

> execute_code

```
from lightgbm import LGBMClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

model = LGBMClassifier(n_estimators=500, learning_rate=0.05,
                        num_leaves=31, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X_train, y_train, cv=cv, scoring='accuracy')
print(f"CV Accuracy: {scores.mean():.4f} (+/- {scores.std()*2:.4f})")
```

Output: CV Accuracy: 0.7847 (+/- 0.0143)

Analysis: 78.5% CV accuracy with low variance across folds indicates stable model. Will generate submission.

ITERATION 4: SUBMISSION AND RESULT

Agent Output (Iteration 4)

Reasoning: Training final model and generating predictions.

> execute_code

```
model.fit(X_train, y_train)
predictions = model.predict(X_test)
submission = pd.DataFrame({'id': test_ids, 'target': predictions})
submission.to_csv('/workspace/submission.csv', index=False)

> submit('/workspace/submission.csv')
```

Result:

Submission successful.
Public leaderboard score: 0.7812
Rank: 1847 / 6823 (73rd percentile)

LogExperiment: name="lgbm_baseline", cv_score=0.7847, approach="LightGBM with imputation"

The trajectory shows the typical pattern: iteration 1 explores, iteration 2 fixes bugs, iterations 3+ optimize. The final CV score (0.7847) closely matches the leaderboard score (0.7812), indicating sound validation methodology.

E.2. Multi-Agent Coordination Failure

This example illustrates a coordination failure between Explorer and Builder agents, demonstrating why multi-agent systems underperform single-agent baselines in our experiments.

Explorer Output

Analysis Summary:

```
{
  "problem_type": "multi-class_classification",
  "num_classes": 120,
  "data_size": {"train": 10222, "test": 5127},
  "key_insight": "Severe class imbalance - 47 classes have <50 samples",
  "recommendation": "Use class-weighted loss or oversampling"
}
```

File written to: /workspace/strategy.json

Builder Attempt

Reading strategy document...

```
> read('/workspace/analysis.json')
```

Error: FileNotFoundError: /workspace/analysis.json not found

Reasoning: Explorer must not have completed. Proceeding with own analysis...

[Agent spends 2.5 hours duplicating data exploration]

The failure occurred because the Explorer wrote to `strategy.json` while the Builder expected `analysis.json`. This filename mismatch caused the Builder to waste time on redundant analysis. More critically, the Builder never received the class imbalance warning and used default settings, achieving only 41st percentile compared to 67th percentile for the single-agent baseline on the same competition.

Root cause analysis: Multi-agent handoffs require exact specification agreement. Our prompts specified output filenames, but agents occasionally deviated. Stricter enforcement (hard-coded paths, schema validation) might mitigate this, but adds system complexity.

E.3. Iterative Recovery from GPU Error

This trajectory demonstrates how iteration enables recovery from resource errors that would otherwise cause complete failure.

Iteration 1: GPU Out-of-Memory

Reasoning: Implementing vision model with pretrained EfficientNet.

```
> execute.code
```

```
model = EfficientNetB4(weights='imagenet', include_top=False)
# ... training code ...
model.fit(train_generator, epochs=50, validation_data=val_generator)
```

Error:

```
RuntimeError: CUDA out of memory. Tried to allocate 2.8 GB
(GPU 0; 80 GB total; 77.2 GB already allocated)
```

Analysis: Batch size too large for available GPU memory.

Iteration 2: Successful Adjustment

Reasoning: Reducing batch size and using mixed precision to fit in memory.

```
> execute_code
# Reduced batch size: 32 -> 8
# Added mixed precision training
from tensorflow.keras.mixed_precision import set_global_policy
set_global_policy('mixed_float16')

model.fit(train_generator, epochs=50, validation_data=val_generator,
          batch_size=8)
```

Output: Training complete. Validation accuracy: 0.823

Without iteration, this run would have failed with zero score. With iteration, the agent diagnosed the memory constraint from the error message and implemented an appropriate fix. This pattern—resource errors followed by successful adjustment—appears frequently in the sampled trajectories and represents one source of iteration’s value.

E.4. Memory-Assisted Strategy Selection

This example shows how memory prevents repeated failures by allowing agents to learn from past experiments.

Without Memory (Competition A)

Iteration 1: Trying neural network approach...

Result: Training diverged (loss = NaN after 3 epochs)

Iteration 2: Adjusting learning rate...

Result: Still diverging (loss = NaN after 7 epochs)

Iteration 3: Trying different optimizer...

Result: Training diverged again

[3 iterations wasted on unstable approach before switching to gradient boosting]

With Memory (Competition B)

Querying memory for similar problems...

Memory result: Competition A (similar: tabular, 50+ features, skewed target)

- Neural network: FAILED (training instability, 3 attempts)
- Gradient boosting: SUCCESS (78th percentile)
- Lesson: "High-dimensional sparse tabular data favors tree models"

Reasoning: Memory indicates neural networks failed on similar data. Starting with gradient boosting.

Result: First iteration achieves 74th percentile.

Memory’s aggregate value is modest (+1.4 MLE points and +0.8 K-LIVE percentile points in Table 6) because such direct transfer is relatively rare: most competitions have sufficiently distinct characteristics that past experience provides limited guidance. However, when applicable, memory prevents the specific failure pattern shown above.

E.5. Trajectory Statistics

Table 33 summarizes a manual audit of trajectory patterns from 500 sampled run logs.

In the audited sample, 58% of runs required at least one iteration to fix bugs or handle resource constraints. This helps explain iteration’s dominant effect: without feedback, many such runs would fail completely rather than achieving median performance.

Table 33. Trajectory patterns across sampled runs.

Pattern	Count	% of Runs
Clean execution (no errors)	165	33%
Bug fix in iteration 2	171	34.2%
Resource error + recovery	72	14.4%
Multiple bug fixes required	47	9.4%
Unrecoverable failure	45	9%
Total	500	100%

E.6. Failure Mode Examples

We document common failure patterns to inform future system design.

Premature commitment. The agent commits to an approach in iteration 1 and never reconsiders despite poor results:

“Iteration 5: Continuing to tune XGBoost hyperparameters. CV score: 0.612 (unchanged from iteration 2). Trying max_depth=12...”

The agent spent 8 iterations on diminishing returns instead of trying alternative approaches.

Format errors. The agent produces valid predictions but fails submission due to format mismatch:

“Submission rejected: Column ‘prediction’ not found. Expected columns: [‘id’, ‘target’]”

The agent used prediction instead of target. This occurred in 12.5% of failures despite explicit format instructions.

Overthinking. The agent spends iterations on elaborate preprocessing that doesn’t improve results:

“Iteration 4: Implementing polynomial feature expansion (degree=3) on all 47 features. This will capture non-linear interactions...”

Result: CV score dropped from 0.72 to 0.68 due to overfitting. Reverting...”

These patterns suggest that iteration’s value comes primarily from error correction, not extended exploration. Agents that make good initial choices benefit less from additional iterations than agents that make fixable mistakes.